API Guideline



Table of Contents

1. Introduction **2**

2. API Gateway **2**

2.1. Maintenance time checking 2

2.2. API Key Verification 3

2.3. Account Status 3

2.4. Request Time Limit 3

3. Supported reports **3**

3.1. Win Loss Detail Report 3

3.2. Outstanding Report 5

4. HMAC SHA 256 **5**

4.1. C# 5

4.2. JavaScript 6

5. Sample Request to API Gateway **6**

5.1. C# 6

5.2. JavaScript 7

5.3. Others 9

1. Introduction

We now support APIs for data crawling demand. To use these APIs, you need API Key info (including Key, PassPhrase, and Secret).

After having API Key info, you can get data by requesting API Gateway using the following URL format: **<gateway domain>/api/<function name>**

Within this document, the following terms shall have the meanings set forth below:

In short	Description
[API-Root-URI]	API Gateway URL

2. API Gateway

Every request through API Gateway needs the following parameters in the header:

Name	Type	Description
Key	string	API Key from API Key Management function
PassPhrase	string	HMAC SHA 256 of API Passphrase with API Secret **HMAC SHA 256 value of your PassPhrase*
Signature	string	HMAC SHA 256 of Timestamp with API Secret **Please read **HMAC SHA 256** section to generate this parameter*
Timestamp	string	Unix time string of request time in milliseconds **Value must not be older than 5 seconds*

All requests will be validated with Maintenance time checking, API Key info, Account status, and Request Times Limit.

2.1. Maintenance time checking

- Responses

Code	Description
503	Under Maintenance Time

2.2. API Key Verification

- Responses

Code	Description
400	Some parameters in the header are not provided correctly
401	API Key info provided is not correct ➔ Key not found ➔ Key expired ➔ PassPhrase not matched ➔ Signature is incorrect

2.3. Account Status

- Responses

Code	Description
403	Forbidden ➔ Account is in protected mode ➔ Account is closed ➔ Account is locked

2.4. Request Time Limit

- Responses

Code	Description
429	Call requests too many times in 1 minute

3. Supported reports

3.1. Win Loss Detail Report

- To get the Win/Loss Detail Report by customer name
- URL: [API-Root-URI]/api/WinLossDetail
- Method: POST

- Parameters:

Name	Type	Description
CustName	string	Username of the customer whose report is viewed

- Responses: JSON object

Name	Type	Description
Report	array of object	Array of customer Win/Loss Detail
Total	object	Total Win/Loss Detail
BetCount	int	Number of tickets
Turnover	string	Turnover of customer
MemberWinLoss	string	Win/Loss of Member
MemberComm	string	Commission of Member
MemberTotal	string	Win/Loss and Commission of Member
AgentWinLoss	string	Win/Loss of Agent
AgentComm	string	Commission of Agent
AgentTotal	string	Win/Loss and Commission of Agent
MasterWinLoss	string	Win/Loss of Master
MasterComm	string	Commission of Master
MasterTotal	string	Win/Loss and Commission of Master
SuperWinLoss	string	Win/Loss of Super
SuperComm	string	Commission of Super
SuperTotal	string	Win/Loss and Commission of Super
Company	string	= – (SuperTotal + MasterTotal + AgentTotal + MemberTotal)

3.2. Outstanding Report

- To get the Outstanding report by customer name
- URL: [API-Root-URI]/api/Outstanding
- Method: POST
- Parameters:

Name	Type	Description
CustName	string	Username of the customer whose report is viewed

- Responses: JSON object

Name	Type	Description
Report	array of object	Array of customer Outstanding
Total	object	Total Outstanding
MemberOutstanding	string	Outstanding of Member
AgentOutstanding	string	Outstanding of Agent
MasterOutstanding	string	Outstanding of Master
SuperOutstanding	string	Outstanding of Super
CustName	string	Username of the customer whose Outstanding value is shown

4. HMAC SHA 256

4.1. C#

```
var hmac = new HMACSHA256(Encoding.UTF8.GetBytes(apiSecret));
var signatureBytes = hmac.ComputeHash(Encoding.UTF8.GetBytes(timestamp));
var signature = Convert.ToBase64String(signatureBytes, 0, signatureBytes.Length);
```

4.2. JavaScript

```
async function HMAC(key, message){
  const g = str => new Uint8Array([...unescape(encodeURIComponent(str))].map(c =>
c.charCodeAt(0))),
  k = g(key),
  m = g(message),
  c = await crypto.subtle.importKey('raw', k, { name: 'HMAC', hash: 'SHA-256' },true, ['sign']),
  s = await crypto.subtle.sign('HMAC', c, m);
  return btoa(String.fromCharCode(...new Uint8Array(s)))
}

HMAC("bd3bd829518f118bee3b99fde03770d8", "1672678800000").then(e =>
console.log(e));
```

5. Sample Request to API Gateway

This is a sample request for an Outstanding report

5.1. C#

Step 1: Get **Key, PassPhrase, Secret,** and **Service URL**.

Step 2: In the sample code below, update **API Key info**, **customer name**, **[API-Root-URI]**, create your application in C#, and call the **SendRequest** function.

```
public void SendRequest()
      {
         var key = "1f02453805a5d36c84b0677e";
         var secret = "your-secret";
         var passPhrase = "your-passphrase";
         var custName = "viewed-name";
         const string apiGateway = "https://[API-Root-URI]/api/Outstanding";

         var hmac = new
System.Security.Cryptography.HMACSHA256(System.Text.Encoding.UTF8.GetBytes(secret));
         var timestamp = DateTimeOffset.Now.ToUnixTimeMilliseconds().ToString();
         var signatureBytes = hmac.ComputeHash(System.Text.Encoding.UTF8.GetBytes(timestamp));
         var signature = Convert.ToBase64String(signatureBytes, 0, signatureBytes.Length);

         ServicePointManager.Expect100Continue = true;
```

```
        ServicePointManager.SecurityProtocol = SecurityProtocolType.Tls12;
        ServicePointManager.ServerCertificateValidationCallback = delegate { return true; };

        using (var client = new System.Net.Http.HttpClient())
        {
            client.DefaultRequestHeaders.Add("Key", key);
            client.DefaultRequestHeaders.Add("PassPhrase", passPhrase);
            client.DefaultRequestHeaders.Add("Signature", signature);
            client.DefaultRequestHeaders.Add("Timestamp", timestamp);

            var param = new OutstandingRequest { CustName = custName };
            var httpContent = new System.Net.Http.StringContent(JsonConvert.SerializeObject(param),
UnicodeEncoding.UTF8, "application/json");
            client.DefaultRequestHeaders.Accept.Add(new
System.Net.Http.Headers.MediaTypeWithQualityHeaderValue("application/json"));
            client.DefaultRequestHeaders.TryAddWithoutValidation("Content-Type", "application/json");

            var response = client.PostAsync(apiGateway, httpContent)
                .GetAwaiter()
                .GetResult()
                .Content.ReadAsStringAsync()
                .GetAwaiter()
                .GetResult();

            System.Diagnostics.Debug.Write(JsonConvert.SerializeObject(response));
        }
    }
```

With OutstandingRequest class

```
public class OutstandingRequest
  {
    public string CustName { get; set; }
  }
```

5.2. JavaScript

Step 1: Get **Key, PassPhrase, Secret,** and **Service URL**.

Step 2: Update **API Key info**, **customer name**, and **[API-Root-URI]** to the sample code below.

```
var key = "1f02453805a5d36c84b0677e";
var secret = "your-secret";
var passPhrase = "your-passphrase";
var custName = "viewed-name";
var apiURL = "https://[API-Root-URI]/api/Outstanding";
```

Step 3: Copy and run the sample code on your web browser console (such as Firefox)



```
» var key = "1f02453805a5d36c84b0677e";
  var secret = "your-secret";
  var passPhrase = "your-passphrase";
  var custName = "viewed-name";
  var apiURL = "https://[API-Root-URI]/api/Outstanding";

  async function HMAC(key, message){
    const g = str => new Uint8Array([...unescape(encodeURIComponent(str))].map(c => c.charCodeAt(0))),
    k = g(key),
    m = g(message),
    c = await crypto.subtle.importKey('raw', k, { name: 'HMAC', hash: 'SHA-256' },true, ['sign']),
    s = await crypto.subtle.sign('HMAC', c, m);
    return btoa(String.fromCharCode(...new Uint8Array(s)))
  }

  function sendRequest(signature) {
      var xhr = new XMLHttpRequest();
      xhr.open("POST", apiURL, true);
      xhr.setRequestHeader('Content-Type', 'application/json;charset=UTF-8');
      xhr.setRequestHeader('Key', key);
      xhr.setRequestHeader('PassPhrase', passPhrase);
      xhr.setRequestHeader('Signature', signature);
      xhr.setRequestHeader('Timestamp', timestamp);

      xhr.onreadystatechange = (data) => {
       if(xhr.responseText) { console.log(xhr.responseText); }
      };

      xhr.send(JSON.stringify({ custName: custName }));
  }


  var timestamp = new Date().getTime().toString();
  HMAC(secret, passPhrase).then(
      _ => HMAC(secret, timestamp).then(e => sendRequest(e));
  );
```

Here is the full sample code:

```
var key = "1f02453805a5d36c84b0677e";
var secret = "your-secret";
var passPhrase = "your-passphrase";
var custName = "viewed-name";
var apiURL = "https://[API-Root-URI]/api/Outstanding";

async function HMAC(key, message){
  const g = str => new Uint8Array([...unescape(encodeURIComponent(str))].map(c =>
c.charCodeAt(0))),
  k = g(key),
```

```
  m = g(message),
  c = await crypto.subtle.importKey('raw', k, { name: 'HMAC', hash: 'SHA-256' },true, ['sign']),
  s = await crypto.subtle.sign('HMAC', c, m);
  return btoa(String.fromCharCode(...new Uint8Array(s)))
}

function sendRequest(signature) {
    var xhr = new XMLHttpRequest();
    xhr.open("POST", apiURL, true);
    xhr.setRequestHeader('Content-Type', 'application/json;charset=UTF-8');
    xhr.setRequestHeader('Key', key);
    xhr.setRequestHeader('PassPhrase', passPhrase);
    xhr.setRequestHeader('Signature', signature);
    xhr.setRequestHeader('Timestamp', timestamp);

    xhr.onreadystatechange = (data) => {
            if(xhr.responseText) { console.log(xhr.responseText); }
    };

    xhr.send(JSON.stringify({ custName: custName }));
}

var timestamp = new Date().getTime().toString();
HMAC(secret, timestamp).then(e => sendRequest(e));
```

5.3. Others

You can use other coding languages or tools (Postman, etc.) to send requests to API Gateway with the appropriate parameters described in this document.